Tienda con Dolibarr: modelo de datos, pseudocódigo y flujogramas

1. Qué cubre este documento

Este documento reúne lo que pide la tarjeta **PE-01** y la **Fase 4** del ROADMAP: el modelo de datos sacado de las capturas de Dolibarr y, para cada programa, su **algoritmo**, su **pseudocódigo** y su **flujograma**. El código en C# (consola, Visual Studio) está en **7 archivos, uno por programa** (de `Programa1_CrearCliente.cs` a `Programa7_CodigoBarrasFactura.cs`); cada uno es independiente, valida los datos y trae pruebas con resultados esperados.

Subtarea de Trello	Programa
Entradas, salidas por orden de llegada (PEPS) y productos perdidos o dañados	Programa 3
Alertas de stock de 5 o menos y vencimiento dentro de 5 días	Programa 4
Descuentos y cantidad mínima para precio mayorista	Programa 5
Cotizaciones (presupuestos)	Programa 5
Reservas y devolver el stock cuando se cancela	Programa 6
Lectura de código de barras, suma de repetidos, total y cambio	Programa 7
Operaciones principales: crear cliente, registrar producto	Programas 1 y 2

Atributos visibles en las capturas y su tipo

Entidad	Atributos (de las capturas)	Tipo de dato
Tercero	Código cliente (CU2610-00102), nombre, teléfono, email, dirección, naturaleza (C), comercial (Tienda Web), estado (Activo)	texto; estado = lógico
Producto	Ref (BEB-001), nombre, precio sin IVA y con IVA, tipo y valor del código de barras (EAN13), en venta, en compra, gestión de stock, categoría, almacén (TIENDA)	texto; precios = decimal; indicadores = lógico
Presupuesto	Ref (COT-0065), cliente, fecha, fecha fin de validez, estado, base imponible, IVA, importe total	texto; fechas; decimal
Línea	Producto, IVA (13%), P.U., P.U. con IVA, cantidad, descuento %, base imponible	decimal; cantidad = entero
Factura	Ref (FA2610-0006), cliente, fecha de facturación, fecha límite de pago, estado (Pendiente de pago), base, IVA, total, ya pagado, resta por pagar, pedido relacionado	texto; fechas; decimal

Relaciones: un tercero tiene muchos presupuestos y muchas facturas; un presupuesto o una factura tiene muchas líneas; un producto aparece en muchas líneas y tiene muchos lotes, movimientos y reservas.

3. Programas

Reglas usadas en todos: IVA 13%; los precios se guardan **sin IVA** y se redondean a 2 decimales; el dinero se maneja con `decimal`.

Programa 1: Crear cliente (Terceros)

Archivo de código: `Programa1_CrearCliente.cs`

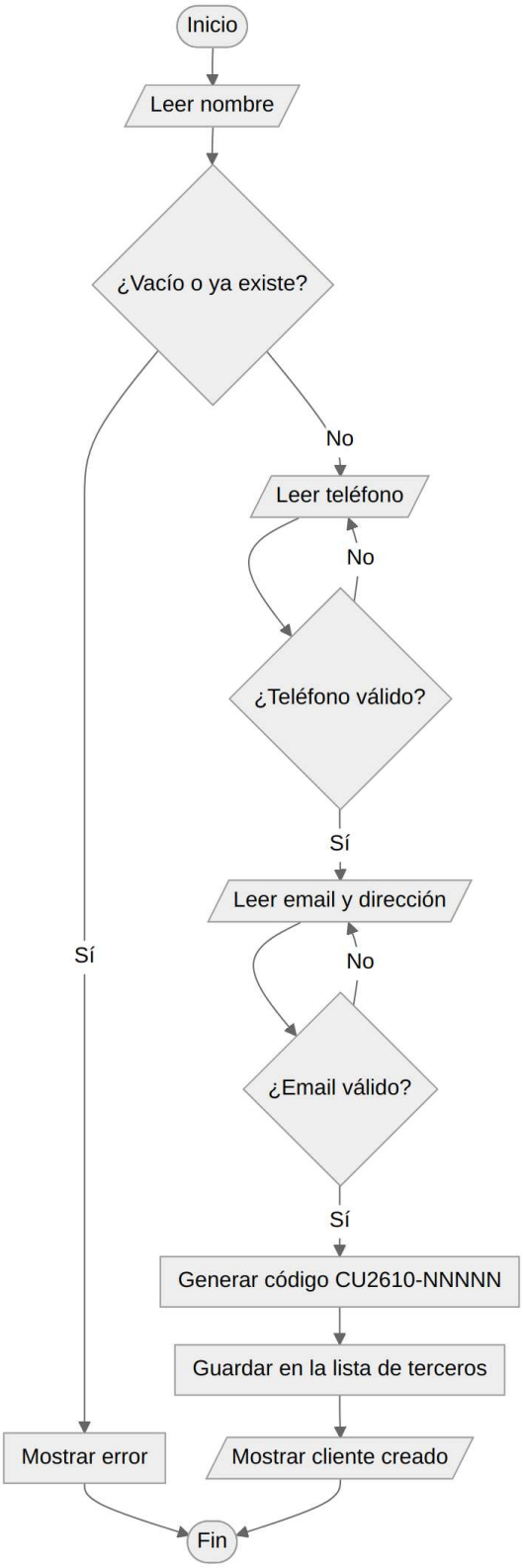
Algoritmo

1. Pedir el nombre; si está vacío o ya existe, avisar y terminar.
2. Pedir el teléfono y repetir hasta que sea válido (7 a 15 dígitos).
3. Pedir el email (opcional) y validarlo; pedir la dirección (opcional).
4. Generar el código CU2610-NNNNN, guardar el cliente en la lista y mostrarlo.

Pseudocódigo

```
PROCESO CrearCliente
  LEER nombre
  SI nombre = "" O existeTercero(nombre) ENTONCES
     ESCRIBIR "Nombre vacío o repetido"
     TERMINAR
  FIN SI
  REPETIR
     LEER telefono
  HASTA QUE longitud(telefono) ENTRE 7 Y 15 Y solo tenga dígitos (o +
al inicio)
  REPETIR
     LEER email (opcional)
  HASTA QUE email = "" O (contiene "@" Y contiene ".")
  LEER direccion (opcional)
  codigo <- "CU2610-" + siguienteNumero (5 dígitos)
  terceros.AGREGAR(codigo, nombre, telefono, email, direccion, activo
= VERDADERO)
  ESCRIBIR "Cliente creado", codigo
FIN PROCESO
```

Flujograma



Programa 1: Crear cliente

Programa 2: Registrar producto (Productos)

Archivo de código: `Programa2_RegistrarProducto.cs`

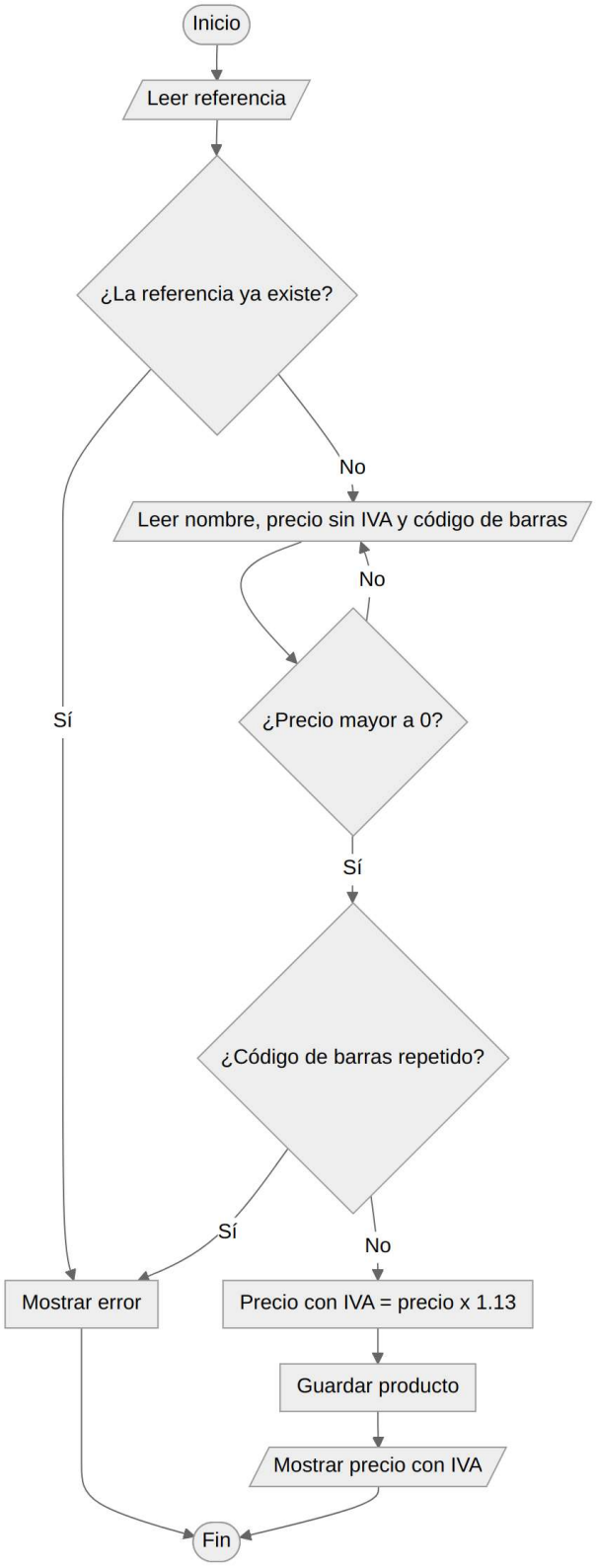
Algoritmo

1. Pedir la referencia; si ya existe, avisar y terminar.
2. Pedir nombre, precio sin IVA (mayor a 0) y código de barras (opcional, no repetido).
3. Calcular el precio con IVA = precio × 1.13.
4. Guardar el producto y mostrar el precio con IVA.

Pseudocódigo

```
PROCESO RegistrarProducto
  LEER ref
  SI existeProducto(ref) ENTONCES
     ESCRIBIR "Referencia repetida"
     TERMINAR
  FIN SI
  LEER nombre
  REPETIR
     LEER precioSinIva
  HASTA QUE precioSinIva > 0
  LEER codigoBarras (opcional)
  SI codigoBarras <> "" Y existeCodigo(codigoBarras) ENTONCES
     ESCRIBIR "Código de barras repetido"
     TERMINAR
  FIN SI
  precioConIva <- REDONDEAR(precioSinIva * 1.13, 2)
  productos.AGREGAR(ref, nombre, precioSinIva, codigoBarras)
  ESCRIBIR "Producto registrado", precioConIva
FIN PROCESO
```

Flujograma



Programa 2: Registrar producto

Programa 3: Entradas y salidas por orden de llegada (PEPS)

Archivo de código: `Programa3_EntradasSalidasPEPS.cs`

Algoritmo

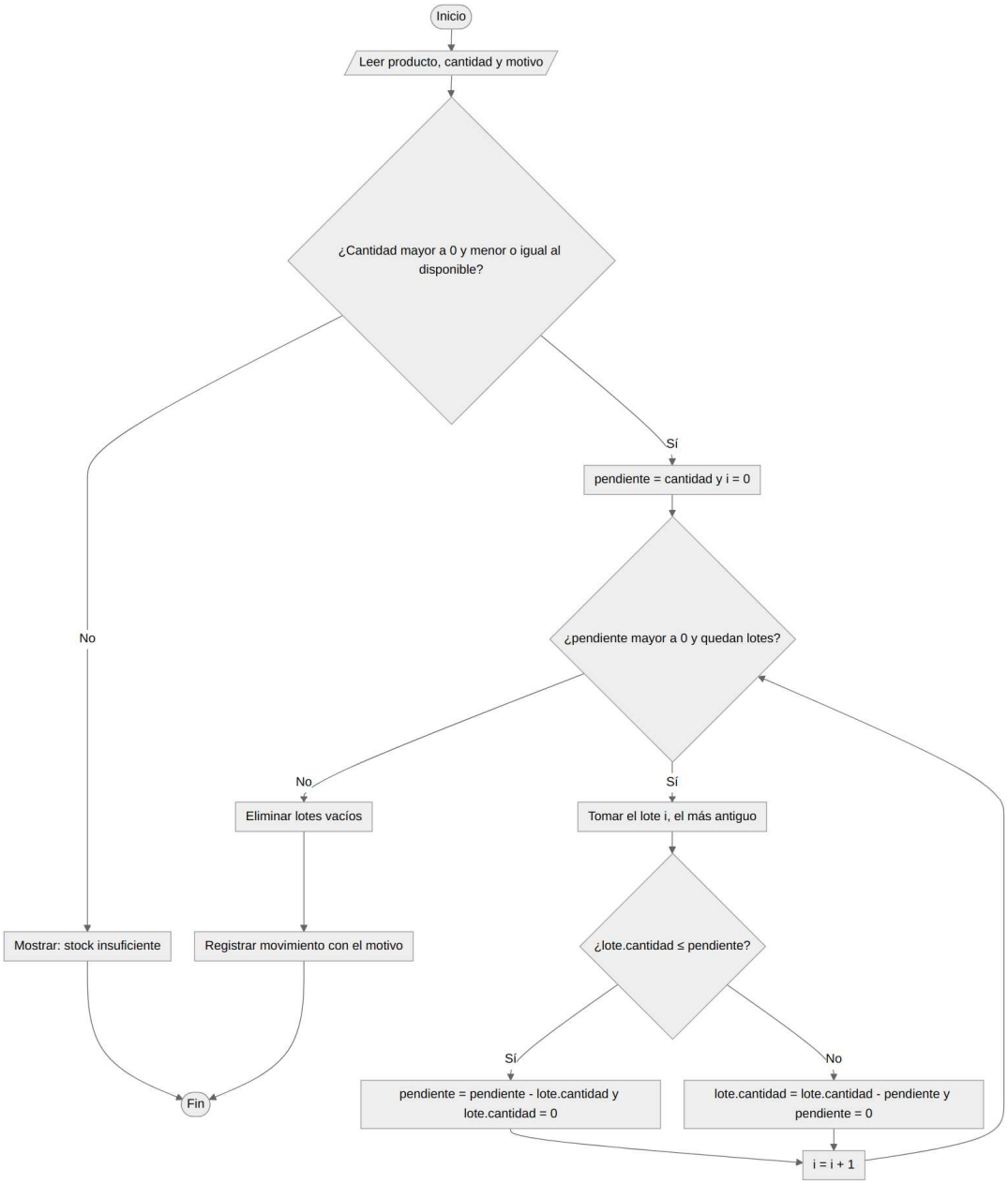
1. **Entrada:** crear un lote con cantidad, fecha de ingreso y vencimiento, y agregarlo al final de la lista (el más antiguo queda primero).
2. **Salida:** si la cantidad es mayor a 0 y no supera el stock disponible, recorrer los lotes desde el primero.
3. Si el lote alcanza para lo pendiente, se descuenta una parte; si no, se consume entero y se pasa al siguiente.
4. Quitar los lotes vacíos y registrar el movimiento con su motivo (venta, pérdida o daño).

Pseudocódigo

```
PROCESO EntradaStock(producto, cantidad, vence)
  SI cantidad <= 0 ENTONCES TERMINAR
  lote <- NUEVO(cantidad, hoy, vence)
  producto.lotes.AGREGAR_AL_FINAL(lote)
  registrar movimiento (+cantidad, "Entrada")
FIN PROCESO

FUNCION SalidaPEPS(producto, cantidad, motivo)
  SI cantidad <= 0 O cantidad > disponible(producto) ENTONCES
     DEVOLVER FALSO
  FIN SI
  pendiente <- cantidad
  i <- 0
  MIENTRAS pendiente > 0 Y i < cantidadDeLotes HACER
     lote <- producto.lotes[i]
     SI lote.cantidad <= pendiente ENTONCES
        pendiente <- pendiente - lote.cantidad
        lote.cantidad <- 0
     SINO
        lote.cantidad <- lote.cantidad - pendiente
        pendiente <- 0
     FIN SI
     i <- i + 1
  FIN MIENTRAS
  ELIMINAR los lotes con cantidad = 0
  registrar movimiento (-cantidad, motivo)
  DEVOLVER VERDADERO
FIN FUNCION
```

Flujograma (salida PEPS)



Programa 3: Salida PEPS

Programa 4: Alertas de stock y vencimiento

Archivo de código: `Programa4_Alertas.cs`

Algoritmo

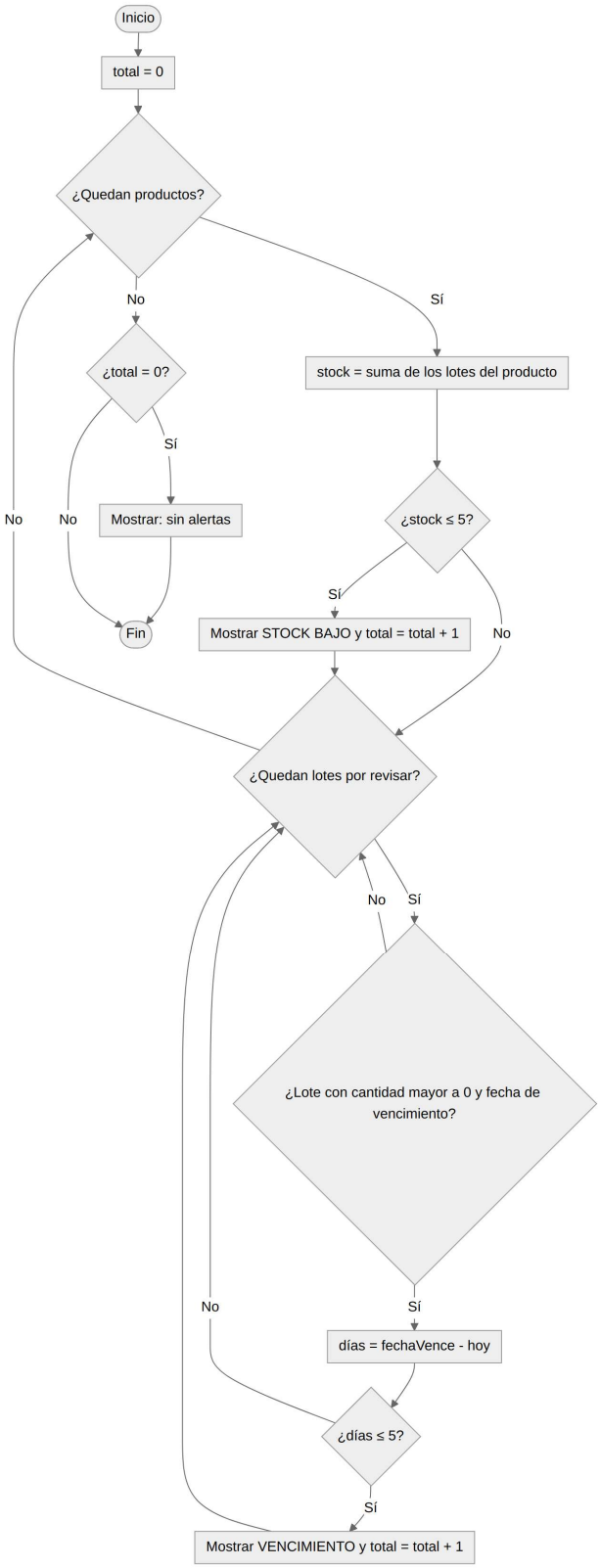
1. Poner el contador de alertas en 0.
2. Para cada producto sumar el stock de sus lotes; si es 5 o menos, mostrar alerta de stock bajo.

3. Para cada lote con cantidad y fecha de vencimiento, calcular los días que faltan; si son 5 o menos, mostrar alerta (o «VENCIDO» si es negativo).
4. Al final, si el contador sigue en 0, mostrar «Sin alertas».

Pseudocódigo

```
FUNCION Alertas
  total <- 0
  PARA CADA producto EN productos HACER
     stock <- suma de las cantidades de sus lotes
     SI stock <= 5 ENTONCES
        ESCRIBIR "STOCK BAJO", producto.ref, stock
        total <- total + 1
     FIN SI
     PARA CADA lote EN producto.lotes HACER
        SI lote.cantidad > 0 Y lote tiene fecha de vencimiento
ENTONCES
           dias <- lote.fechaVence - hoy
           SI dias <= 5 ENTONCES
              ESCRIBIR "VENCIMIENTO", producto.ref, dias
              total <- total + 1
           FIN SI
        FIN SI
     FIN PARA
  FIN PARA
  SI total = 0 ENTONCES ESCRIBIR "Sin alertas"
  DEVOLVER total
FIN FUNCION
```

Flujograma



Programa 4: Alertas

Programa 5: Generar presupuesto (cotización) con descuentos y precio mayorista

Archivo de código: `Programa5_Presupuesto.cs`

Algoritmo

1. Buscar el cliente; si no existe, terminar.
2. Leer productos y cantidades; si un producto se repite, sumar la cantidad en su línea.
3. Para cada línea elegir el descuento: el manual, o 10% si la cantidad llega al mínimo mayorista (24).
4. Calcular la base de cada línea, la base total, el IVA 13% y el total.
5. Poner la fecha de fin de validez (hoy + 15 días), el estado «Validado» y mostrar el presupuesto.

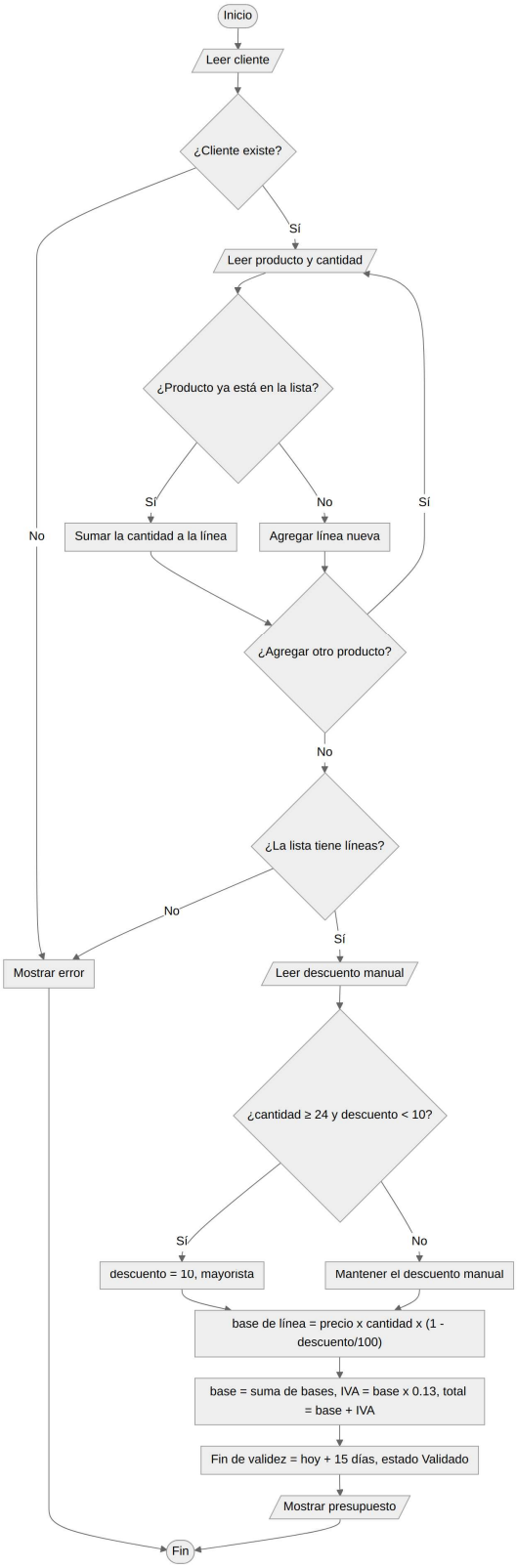
Pseudocódigo

```
FUNCION DescuentoAplicable(cantidad, descuentoManual)
  dto <- descuentoManual
  SI cantidad >= 24 Y dto < 10 ENTONCES
     dto <- 10
  FIN SI
  DEVOLVER dto
FIN FUNCION

PROCESO GenerarPresupuesto
  LEER cliente (código o nombre)
  SI no existeTercero(cliente) ENTONCES TERMINAR
  lineas <- lista vacía
  REPETIR
     LEER producto (ref o código de barras)
     SI existeProducto ENTONCES
        LEER cantidad
        SI el producto ya está en lineas ENTONCES
           sumar cantidad a esa línea
        SINO
           lineas.AGREGAR(producto, precio, cantidad)
        FIN SI
     FIN SI
     LEER otro (s/n)
  HASTA QUE otro = "n"
  SI lineas está vacía ENTONCES TERMINAR
  LEER descuentoManual
  PARA CADA linea EN lineas HACER
     linea.dto  <- DescuentoAplicable(linea.cantidad, descuentoManual)
     linea.base <- REDONDEAR(linea.precio * linea.cantidad * (1 -
linea.dto / 100), 2)
  FIN PARA
  base  <- SUMA(linea.base)
  iva   <- REDONDEAR(base * 0.13, 2)
  total <- base + iva
  fechaFin <- hoy + 15 días
```

```
  estado <- "Validado (presupuesto abierto)"
  guardar y mostrar el presupuesto
FIN PROCESO
```

Flujograma



Programa 5: Generar presupuesto

Programa 6: Reservas (crear y cancelar)

Archivo de código: `Programa6_Reservas.cs`

Algoritmo

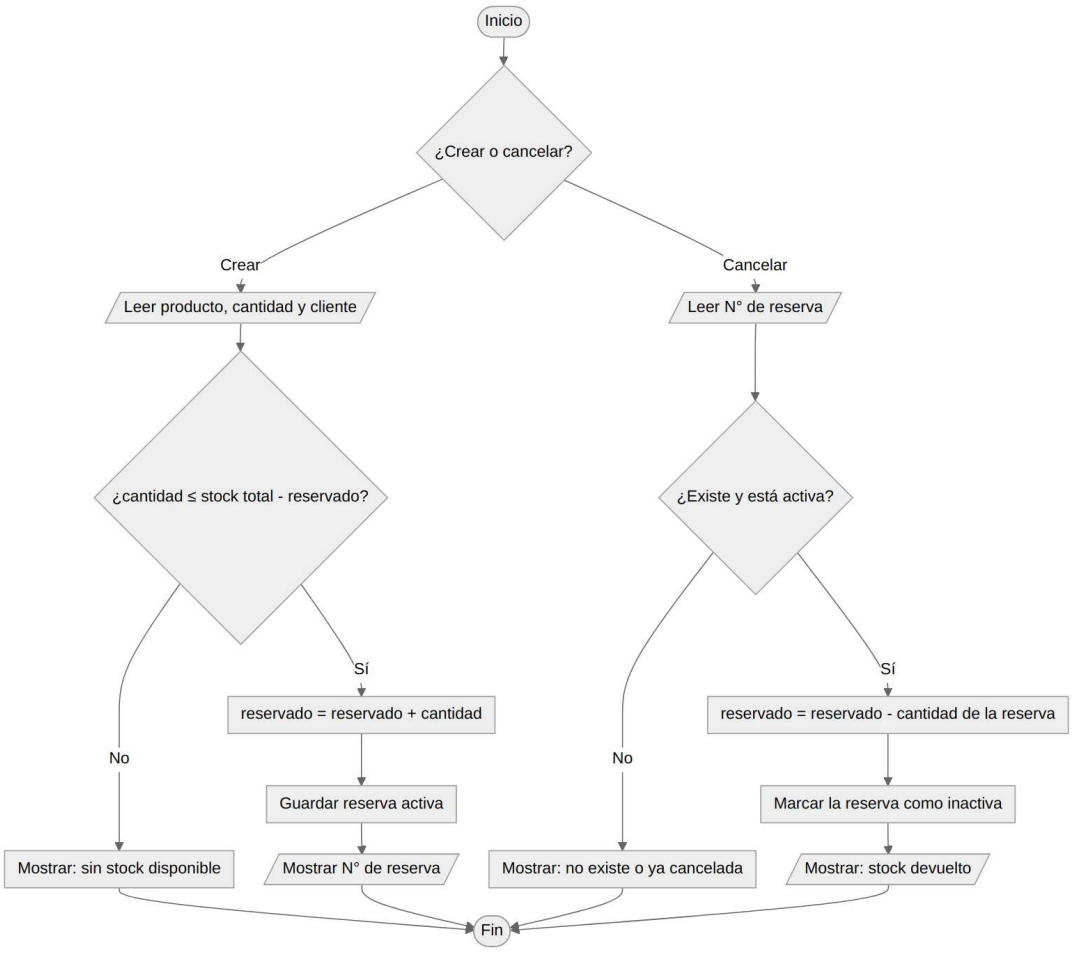
1. **Crear:** si la cantidad es mayor a 0 y no supera (stock total − reservado), sumar la cantidad a «reservado» y guardar la reserva como activa.
2. **Cancelar:** buscar la reserva por número; si existe y está activa, restar su cantidad de «reservado» (el stock vuelve a quedar disponible) y marcarla inactiva.

Pseudocódigo

```
FUNCION CrearReserva(producto, cantidad, cliente)
  SI cantidad <= 0 O cantidad > (stockTotal(producto) -
producto.reservado) ENTONCES
     DEVOLVER nulo
  FIN SI
  producto.reservado <- producto.reservado + cantidad
  reserva <- NUEVA(id, producto.ref, cliente, cantidad, activa =
VERDADERO)
  reservas.AGREGAR(reserva)
  DEVOLVER reserva
FIN FUNCION

FUNCION CancelarReserva(id)
  reserva <- BUSCAR en reservas con ese id Y activa = VERDADERO
  SI no existe ENTONCES DEVOLVER FALSO
  producto <- BUSCAR producto de la reserva
  producto.reservado <- producto.reservado - reserva.cantidad
  reserva.activa <- FALSO
  DEVOLVER VERDADERO
FIN FUNCION
```

Flujograma



Programa 6: Reservas

Programa 7: Emitir factura (venta, total y cambio)

Archivo de código: `Programa7_CodigoBarrasFactura.cs`

Algoritmo

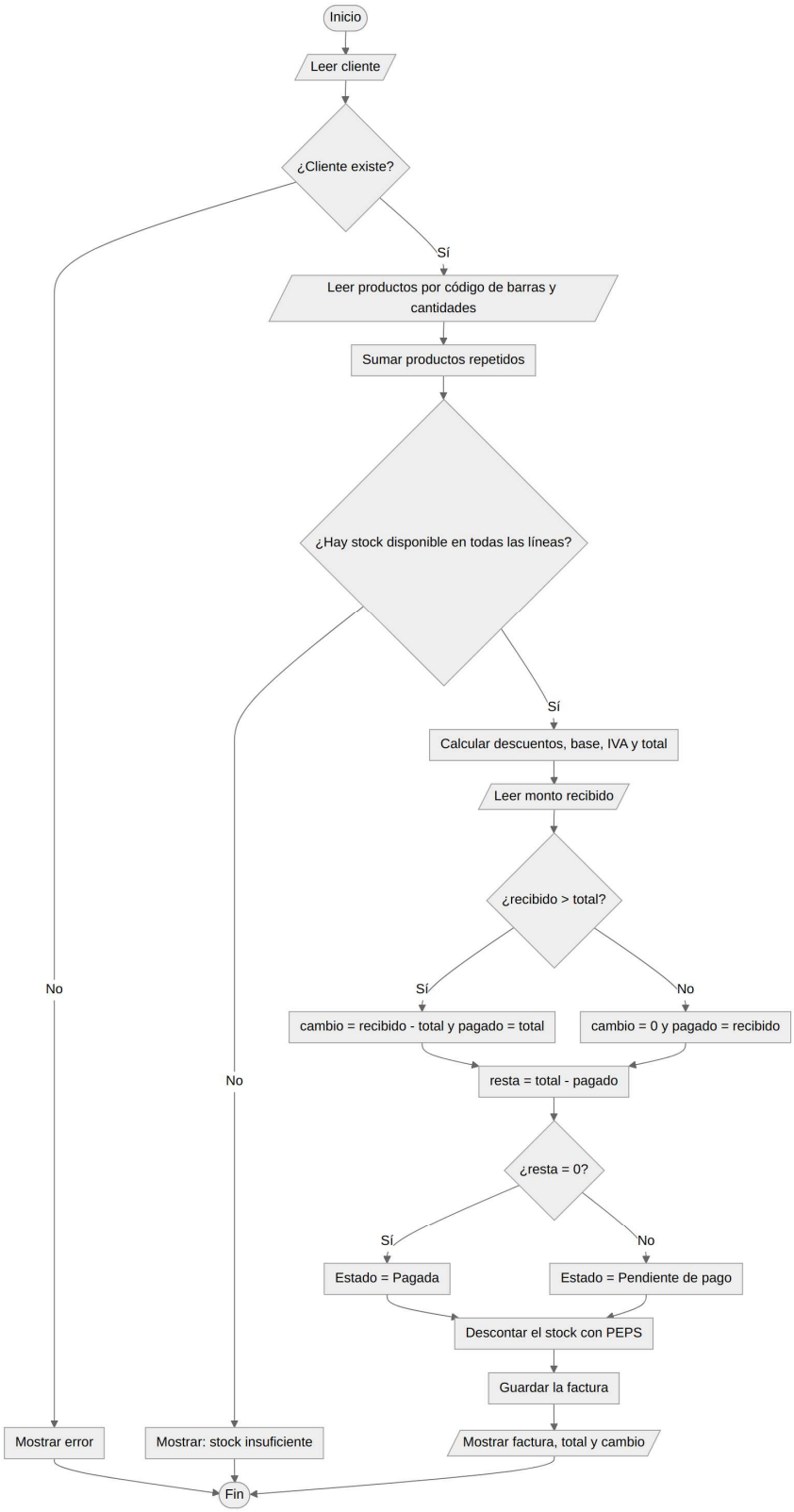
1. Buscar el cliente; leer los productos (por código de barras) y sumar los repetidos.
2. Comprobar que **todas** las líneas tengan stock disponible, antes de descontar nada.
3. Calcular descuentos, base, IVA y total (igual que en el presupuesto).
4. Leer el monto recibido: si es mayor al total, el cambio es la diferencia; la resta por pagar es total − pagado.
5. Si la resta es 0 el estado es «Pagada»; si no, «Pendiente de pago». Descontar el stock con PEPS y guardar.

Pseudocódigo

```
PROCESO EmitirFactura
  LEER cliente
  SI no existeTercero(cliente) ENTONCES TERMINAR
  lineas <- armar líneas (código de barras, suma de repetidos)
```

```
  SI lineas está vacía ENTONCES TERMINAR
  PARA CADA linea EN lineas HACER
     SI disponible(linea.producto) < linea.cantidad ENTONCES
        ESCRIBIR "Stock insuficiente", linea.producto
        TERMINAR
     FIN SI
  FIN PARA
  calcular dto y base de cada línea
  base  <- SUMA(linea.base)
  iva   <- REDONDEAR(base * 0.13, 2)
  total <- base + iva
  LEER recibido
  SI recibido > total ENTONCES
     cambio <- recibido - total
     pagado <- total
  SINO
     cambio <- 0
     pagado <- recibido
  FIN SI
  resta <- total - pagado
  SI resta = 0 ENTONCES estado <- "Pagada"
  SINO estado <- "Pendiente de pago"
  PARA CADA linea EN lineas HACER
     SalidaPEPS(linea.producto, linea.cantidad, "Venta")
  FIN PARA
  facturas.AGREGAR(factura)
  ESCRIBIR factura, total, cambio
FIN PROCESO
```

Flujograma



Programa 7: Emitir factura

4. Datos de ejemplo y resultados esperados

Los casos salen de las capturas de Dolibarr (la opción **Pruebas** del menú de cada programa los ejecuta y marca OK o FALLA). La fecha del sistema en las pruebas es el 19/10/2026.

Caso	Datos de entrada	Resultado esperado
Factura FA2610-0006	Mark Flores; BEB-001 (Chicha de maní con leche 1 L) ×1 y BEB-003 (Kéfir natural 1 L) ×1; recibe 30 Bs	Base 22.12; IVA 2.88; total 25.00; cambio 5.00; estado Pagada
Factura sin pago	YOG-024 (Yogur bebible canela 1 L) leído 2 veces; recibe 0	1 sola línea con cantidad 2; total 24.00; resta por pagar 24.00; Pendiente de pago
Presupuesto COT-0065	Guardería Pequeños Pasos; LEC-011 ×36, YOG-029 ×100, LEC-002 ×24	10% mayorista en las 3 líneas; base 1500.66; IVA 195.09; total 1695.75; válido hasta 03/11/2026
Descuento mayorista	23 u. / 24 u. / 5 u. con 15% manual	0% / 10% / 15%
PEPS	BEB-001 con lotes de 5 y 10; sale 7	Queda 1 lote de 8; una salida de 99 devuelve FALSO
Alertas	Datos iniciales	4 alertas: HEL-006 (2/5), LEC-013 (0/5), YOG-009 (2/5) y un lote de BEB-001 que vence en 4 días
Reserva	YOG-024 con 28 disponibles; reservar 10; cancelar	18 disponibles tras reservar; 28 tras cancelar; cancelar otra vez devuelve FALSO

5. Cómo ejecutar los programas

1. En Visual Studio: **Crear un proyecto nuevo** → **Aplicación de consola (C#)**. Se usa un proyecto por programa.
2. Abrir el archivo **Program.cs**, borrar todo su contenido y pegar el código del archivo del programa que se quiere probar.
3. Presionar **Ctrl+F5**. En el menú, la opción **Pruebas** ejecuta los casos de la sección 4 y marca [OK] o [FALLA]; las demás opciones se usan a mano.

Programa	Archivo	Opción de pruebas
1. Crear cliente	Programa1_CrearCliente.cs	3
2. Registrar producto	Programa2_RegistrarProducto.cs	3
3. Entradas y salidas (PEPS)	Programa3_EntradasSalidasPEPS.cs	5
4. Alertas	Programa4_Alertas.cs	3
5. Presupuesto	Programa5_Presupuesto.cs	3
6. Reservas	Programa6_Reservas.cs	4
7. Código de barras y factura	Programa7_CodigoBarrasFactura.cs	3

6. Supuestos a confirmar con el equipo

- Las capturas no muestran la regla mayorista: se supuso **24 unidades o más = 10%** (coincide con COT-0065). Está en las constantes `MIN_MAYORISTA` y `DTO_MAYORISTA`, fáciles de cambiar.
- Alertas: stock de **5 o menos** y vencimiento en **5 días o menos**, según la tarjeta. La validez del presupuesto es de **15 días** (08/10 a 08/25 en COT-0065).
- En la captura de COT-0065 el IVA aparece en 0.00, pero el total 1,695.75 corresponde a base + 13%, por eso se calculó con IVA.
- Los códigos de barras (salvo YOG-024), los stocks y el código del cliente Guardería Pequeños Pasos son datos de ejemplo inventados para poder probar.